

oldver\data_setup3.py

```
1 import os
2 import json
3 import numpy as np
4 import pandas as pd
5 from PIL import Image
6 from sklearn.model_selection import train_test_split
7
8 from config3 import (
9     seed_everything, SEED, TRAIN_DIR, OUTPUT_DIR, STATS_PATH,
10     CLASS_NAMES, CLASS_TO_IDX, VAL_SPLIT, IMAGE_SIZE,
11 )
12
13
14 def build_dataframe() -> pd.DataFrame:
15     records = []
16     for class_name in CLASS_NAMES:
17         class_dir = TRAIN_DIR / class_name
18         if not class_dir.exists():
19             print(f" [WARNING] Folder not found: {class_dir}")
20             continue
21         files = sorted(
22             f for f in os.listdir(class_dir)
23             if f.lower().endswith((".png", ".jpg", ".jpeg"))
24         )
25         for fname in files:
26             records.append({
27                 "filepath": str(class_dir / fname),
28                 "label": CLASS_TO_IDX[class_name],
29                 "class_name": class_name,
30                 "filename": fname,
31             })
32     return pd.DataFrame(records)
33
34
35 def compute_train_stats(train_df: pd.DataFrame) -> dict:
36     """Mean/std per RGB channel over the TRAIN split only to prevent leakage."""
37     print("\n Computing channel mean/std on TRAIN split ...")
38     psum = np.zeros(3, dtype=np.float64)
39     psum_sq = np.zeros(3, dtype=np.float64)
40     n_pix = 0
41     for fp in train_df["filepath"]:
42         img = Image.open(fp).convert("RGB").resize((IMAGE_SIZE, IMAGE_SIZE))
43         arr = np.asarray(img, dtype=np.float64) / 255.0 # H,W,3 in [0,1]
44         psum += arr.sum(axis=(0, 1))
45         psum_sq += (arr ** 2).sum(axis=(0, 1))
46         n_pix += arr.shape[0] * arr.shape[1]
47     mean = psum / n_pix
48     std = np.sqrt(psum_sq / n_pix - mean ** 2)
49     stats = {"mean": mean.tolist(), "std": std.tolist()}
50     print(f" mean = {np.round(mean, 4).tolist()}")
51     print(f" std = {np.round(std, 4).tolist()}")
52     return stats
53
54
55 def main():
56     seed_everything(SEED)
57     print("=" * 70)
58     print(" DATA SETUP – scan folders, stratified split, leak-safe stats")
59     print("=" * 70)
60
61     df = build_dataframe()
62     print(f"\n Total images : {len(df)}")
63     print(" Class distribution:")
64     for cls in CLASS_NAMES:
65         c = int((df['class_name'] == cls).sum())
66         print(f" {cls} : {c:>4d} ({c / len(df) * 100:.1f} %)")
67
68     # Stratified split – keeps the 600/670/310 ratio inside both sets.
69     train_df, val_df = train_test_split(
70         df, test_size=VAL_SPLIT, random_state=SEED, stratify=df["label"],
```

```

71 )
72 train_df = train_df.reset_index(drop=True)
73 val_df = val_df.reset_index(drop=True)
74
75 train_df.to_csv(OUTPUT_DIR / "train_split.csv", index=False)
76 val_df.to_csv(OUTPUT_DIR / "val_split.csv", index=False)
77 print(f"\n Train : {len(train_df)} Val : {len(val_df)}")
78 print(" Stratification check (val):")
79 for cls in CLASS_NAMES:
80     tot = int((df['class_name'] == cls).sum())
81     v = int((val_df['class_name'] == cls).sum())
82     print(f" {cls} : {v}/{tot} ({v / tot * 100:.1f} % in val)")
83
84 stats = compute_train_stats(train_df)
85 with open(STATS_PATH, "w", encoding="utf-8") as f:
86     json.dump(stats, f, indent=2)
87 print(f"\n Saved: train_split.csv, val_split.csv, train_stats.json -> {OUTPUT_DIR}")
88 print(" Data setup complete.\n")
89
90
91 if __name__ == "__main__":
92     main()

```